

Proof-of-Work, Weight & Accumulated Weight

How Hathor measures computational work as a single number — **weight** — and how summed weight across the DAG decides which history is real.

SUBJECT hathor-core · Part I · Track B (domain concepts)

CHAPTER 09 · Foundations · Concepts

BRANCH study/hathor-core-studies

EDITION 2026 · v1

Nonce · Target · Weight ($\log_2$ work) · Difficulty adjustment (DAA) · Accumulated weight · Score · Heaviest-DAG rule · Logarithms

Table of Contents

Chapter 9 — Proof-of-Work, Weight & Accumulated Weight	3
9.1 Proof-of-work, refreshed and made concrete	3
9.2 A two-minute primer on logarithms	4
9.3 Weight: work as a single number	5
9.4 Accumulated weight and score: summing work across the DAG	6
9.5 Keeping the rate steady: the difficulty adjustment algorithm	6
9.6 The heaviest-DAG rule	7
Recap	8

Chapter 9 — Proof-of-Work, Weight & Accumulated Weight

What you'll learn in this chapter

- How proof-of-work is actually checked: the **nonce** → **hash** → **target** relationship, refreshed from Chapter 6 and made concrete.
- Why Hathor measures work as a **weight** — a single float that is the base-2 logarithm of the number of hash attempts — and why a logarithm is the right unit.
- What the **difficulty adjustment algorithm (DAA)** does: tuning weight so blocks arrive at a steady rate, and giving transactions a minimum weight.
- **Accumulated weight** and **score**: how work sums up across the DAG, and how that sum, not chain length, decides which history wins.
- The **heaviest-DAG rule** — Hathor's generalization of "most-work-wins" — and a **bridge** to the verifier, the DAA, and the consensus code.

Chapter 6 established why proof-of-work exists (Sybil resistance, costly history-rewriting); Chapter 8 established the structure work has to secure (a DAG, not a chain). This chapter joins them: how Hathor quantifies work, and how those quantities, accumulated across the graph, answer "which version of history is real." The central idea — measuring work as a logarithm called weight — is one of Hathor's clean departures from Bitcoin, and once you see why a logarithm, the rest follows.

A short primer on logarithms is included (§9.2) because the whole chapter rests on one, and the book does not assume you carry that comfortably. Track B cites Hathor code; this chapter does, at the points where the concept becomes a concrete function.

9.1 Proof-of-work, refreshed and made concrete

Recall the mechanism from Chapter 6. To produce a valid vertex, a miner must find a **nonce**¹ — a number — such that hashing the vertex together with that nonce yields a hash below a **target**². Because the hash function is one-way and avalanche (Chapter 6 §6.4), there's no shortcut: the miner can only try nonce after nonce, billions per second, until one produces a small-enough hash. Finding it is hard; checking it is one hash.

In code, this is exactly what the verifier does. `verify_pow` (`hathor/verification/vertex_verifier.py:142`) computes the vertex's hash and checks it is numerically below the target. The target itself is derived from the vertex's weight by `get_target` (`base_transaction.py:361`), which computes — conceptually — `target = 2^(256 - weight) - 1`. Read that relationship slowly: a 256-bit hash is a number between 0 and $2^{256}-1$; requiring it to fall below $2^{(256-\text{weight})}$ means only a $1 / 2^{\text{weight}}$ fraction of all possible hashes qualify. So:

The core relationship. A vertex of weight w requires, on average, about 2^w hash attempts to find a valid nonce. Higher weight $\Rightarrow$ smaller target $\Rightarrow$ rarer valid hash $\Rightarrow$ more work. The weight is the difficulty, expressed as an exponent.

That last phrase is the key to the whole chapter. Bitcoin stores difficulty as a compact encoding of the target — a 256-bit threshold. Hathor stores it as the exponent instead: a single floating-point number, the weight. To see why that's the better unit, we need the logarithm.

9.2 A two-minute primer on logarithms

A **logarithm**³ answers the question: "to what power must I raise the base to get this number?" In base 2 — the natural base when we're counting doublings — `log2(x)` asks "2 to the what equals x?"

$2^{10} = 1024$	so	$\log_2(1024) = 10$
$2^{20} = 1,048,576$	so	$\log_2(1,048,576) = 20$
$2^{21} = 2,097,152$	so	$\log_2(2,097,152) = 21$

Two properties are all you need:

1. **A logarithm turns huge numbers into small, manageable ones.** The number of hash attempts to mine a block is astronomically large — quadrillions and up. Its base-2 logarithm is a tidy number like 60 or 76. Weight lives on the logarithmic scale, so it stays small and readable while the underlying work is gigantic.
2. **Adding logarithms multiplies the underlying numbers.** `log2(a) + log2(b) = log2(a × b)`. Equivalently, adding one to a weight doubles the work: weight 21 is twice the work of weight 20, and weight 22 is twice again. Each `+1` of weight is a doubling. This is why weights can be summed to combine work — a property §9.4 relies on completely.

With those in hand, weight has a precise meaning: **weight = $\log_2(\text{number of hash attempts})$** . A block of weight 60 represents about $2^{60} \approx 10^{18}$ attempts; a block of weight 61 represents twice that.

9.3 Weight: work as a single number

So Hathor attaches to every vertex a field `weight`, a `float` (`base_transaction.py:201`), and treats it as the logarithm of the work the vertex embodies. The conversions are explicit in `hathor/utils/weight.py`:

```
def weight_to_work(weight: float) -> int:    # how many attempts a weight represents
    return floor(0.5 + 2 ** weight)          # ≈ 2^weight    (utils/weight.py:18)

def work_to_weight(work: int) -> float:      # the inverse
    return log2(work)                        # (utils/weight.py:23)
```

Why is a single float a better representation than Bitcoin's target?

- **It's readable and comparable.** "This block has weight 62" is immediately meaningful, and comparing two weights is comparing two floats. No unpacking a compact threshold.
- **It's additive.** Because adding weights multiplies work (§9.2), the total work behind a region of the DAG is almost just a sum of weights — which is exactly what consensus needs to do constantly (§9.4). Summing thresholds directly would be meaningless; summing logarithms is natural.
- **It's smooth.** Weight is a continuous float, so difficulty can be tuned in fine increments rather than discrete steps — useful for the steady adjustment in §9.5.

The trade-off is small: floating-point weight needs care to compare and store precisely, which is why the code converts to integer work (`weight_to_work`) when it needs exact arithmetic, and keeps weight for the human-facing and per-vertex value. Internally, accumulated work is tracked as an integer (`accumulated_weight: int`) and converted as needed.

Transactions have weight too. In a chain, only blocks are mined. In Hathor's DAG, every vertex — transaction included — carries proof-of-work, because a transaction attaches itself directly to the ledger (Chapter 8) and must pay some cost to do so (an anti-spam measure: making each transaction cost a little work deters flooding the DAG). Transaction weights are much smaller than block weights, but they are real, and they contribute to the accumulated total.

9.4 Accumulated weight and score: summing work across the DAG

Here is where the DAG (Chapter 8) and weight (this chapter) combine into consensus. A single vertex's weight is the work to make that one vertex. But security comes from all the work piled behind a vertex — every later vertex that confirms it (Chapter 8 §8.5). Hathor captures this with two metadata fields, stored on each vertex's

`TransactionMetadata` (`hathor/transaction/transaction_metadata.py`):

- **accumulated_weight** (`transaction_metadata.py:48`): the total work of a vertex plus all the work that confirms it through the graph. As more vertices attach behind it, its accumulated weight grows. This is the DAG generalization of Chapter 6's "blocks built on top" — except instead of counting blocks, Hathor sums their work.
- **score** (`transaction_metadata.py:49`): the metric used to compare competing block chains. For a block, the score reflects the accumulated work of the best chain up to that block. The chain of blocks with the **highest score** is the canonical one.

The reason weight's additivity (§9.2) matters now becomes concrete: to know how much work stands behind a vertex, the node sums the weights (as work) reachable from it. That sum is only meaningful because each `+1` of weight is a consistent doubling — logarithms are the unit that makes "total work in this region of the graph" a well-defined, computable number.

accumulated weight = this vertex's work + all confirming work behind it

(tx, w=14) ← (tx, w=15) ← (block, w=60) ← (block, w=61)

↑
└────────── accumulated weight of the leftmost tx ─────────┘
grows as each new confirming vertex adds its work

9.5 Keeping the rate steady: the difficulty adjustment algorithm

If weight were fixed, two problems would follow: as more miners join, blocks would come faster and faster (more hashing power finds valid nonces sooner); and the network couldn't hold a predictable rhythm. Every proof-of-work system therefore adjusts difficulty over time. Hathor's lives in `hathor/daa.py` — the **difficulty adjustment algorithm (DAA)**⁴ .

The goal is a steady **average time between blocks** — `AVG_TIME_BETWEEN_BLOCKS = 30` seconds (a settings constant from the hathorlib base, `:156`). The algorithm `calculate_next_weight` (`daa.py:94`) works on the principle every DAA shares:

- Look back over a window of recent blocks and measure how long they actually took to find.
- If they came **faster** than the 30-second target, the network has gained hashing power → **raise** the weight (more work required) to slow back down.
- If they came **slower**, **lower** the weight to speed back up.

It's a feedback loop: weight chases the target block time, nudged up or down by recent reality. (There's also a weight decay safeguard that lowers difficulty if blocks stall badly, so the chain can't freeze if hashing power suddenly drops.)

Transactions get a related treatment: `minimum_tx_weight` (`daa.py:176`) computes a floor on a transaction's weight based on its size and amount — bigger transactions must do a little more work — which is the anti-spam mechanism of §9.3 made precise.

9.6 The heaviest-DAG rule

Now Chapter 6's "most-work-wins" can be stated in Hathor's terms. In a chain, the rule is loosely "longest chain," precisely "most cumulative work." In Hathor's DAG, it is:

The history with the greatest accumulated weight wins. When there are competing versions of history — two block chains, or two conflicting transactions — the node prefers the one with more total work behind it, measured by score / accumulated weight.

This is not "the longest" or "the one with the most vertices" — a few heavy vertices can outweigh many light ones, exactly as Chapter 6 warned. It is total work, summed via weight. Because work cannot be faked (only paid for in hashing), preferring the heaviest history means an attacker wanting to rewrite the past must out-compute everyone confirming the real past — the Chapter 6 security argument, now operating over a graph instead of a line.

What happens when the heaviest history changes — when a competing branch overtakes the current one, or two transactions collide on an input — is the resolution mechanism: the losing side is **voided**. That is Chapter 10, which closes Track B by turning "the heaviest wins" into the concrete rules for conflicts, reorgs, and finality.

Recap

Concept	Meaning	In Hathor
Nonce	the number a miner varies	searched until hash < target
Target	threshold the hash must beat	<code>get_target</code> , $\approx 2^{(256-\text{weight})}$ (:361)
Weight	$\log_2(\text{hash attempts})$ — work as one float	<code>weight</code> field (:201); +1 = double work
weight $\leftrightarrow$ work	convert log $\leftrightarrow$ count	<code>utils/weight.py:18,23</code>
DAA	tune weight for steady block time	<code>daa.py:94</code> ; target 30 s (:156)
min tx weight	per-transaction work floor (anti-spam)	<code>daa.py:176</code>
Accumulated weight	a vertex's work + all confirming work	<code>metadata</code> :48
Score	metric to compare chains	<code>metadata</code> :49
Heaviest-DAG rule	most accumulated weight wins	the consensus preference

Hathor measures proof-of-work as **weight** — the base-2 logarithm of the number of hash attempts — because a logarithm keeps an astronomically large quantity small and readable, and because adding weights corresponds to multiplying work, which lets the node sum work across the DAG. Every vertex, block or transaction, carries weight; the difficulty algorithm tunes block weight toward a 30-second rhythm and sets a per-transaction floor against spam; and the work behind any vertex accrues as **accumulated weight**, with the heaviest history winning. The one question left is operational: when two histories compete — a reorg, or two transactions spending the same coin — exactly how does the loser get marked and removed, and when is anything ever final? That is **voiding, conflicts, and finality**, the close of Track B.

1. A nonce ("number used once") is the field a miner repeatedly changes so each attempt re-hashes the vertex to a different value, searching for one whose hash falls below the target. $\leftrightarrow$
2. A target is the numeric threshold a vertex's hash must fall below to be valid proof-of-work. In Hathor it is derived from the vertex's weight: roughly $2^{(256 - \text{weight})}$, so higher weight means a smaller target and more work. $\leftrightarrow$

3. A logarithm $\log_b(x)$ is the exponent to which the base b must be raised to obtain x (e.g. $\log_2(1024) = 10$). Logarithms compress large numbers and convert multiplication into addition. Hathor's weight is a base-2 logarithm of work. ↩
4. The difficulty adjustment algorithm (DAA) periodically recomputes the required weight so that blocks are found at a roughly constant average rate despite changing total hashing power. Hathor's is in `hathor/daa.py`. ↩